COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/02/2026	**DATE SUBMITTED:**	10/02/2026
NAME:	Julian Benedict D.R. Barizo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 [Graphs]

1. PROCEDURE OUTPUT

ILO A	
Code Output	PS C:\Users\TIPQC\Documents\Barizo_CPE010\HOA10p1> Graph adjacency list (start_vertex, end_vertex, weight): (0, 2, 4) (0, 1, 2) (1, 4, 3) (2, 3, 2) (3, 1, 4) (4, 3, 3)
ILO B1	
Code Output	PS C:\Users\TIPQC\Documents\Barizo_CPE010\HOA10p1> & C:\Users\TIPQC\Documents\Barizo_CPE010\HOA10p1\ILOB1.exe 1: {2: 0}, {5: 0}, 2: {1: 0}, {5: 0}, {4: 0}, 3: {4: 0}, {7: 0}, 4: {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0}, 5: {1: 0}, {2: 0}, {4: 0}, {8: 0}, 6: {4: 0}, {7: 0}, {8: 0}, 7: {3: 0}, {6: 0}, 8: {4: 0}, {5: 0}, {6: 0}, 8: {4: 0}, {5: 0}, {6: 0}, DFS Order of vertices: 1 5 8 6 7 3 4 2

Code Snippet

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

#ifndef ILOB1_H
#define ILOB1_H

template <typename T>
    class Graph;

template <typename T>
    struct Edge{

        size_t src;
        size_t dest;
        T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
        inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
        inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
    os << i << ":\t";
    auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }

return os;
}

template <typename T>
class Graph
{
public:
```

```cpp
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {}

    // Return number of vertices in the graph
    auto vertices() const{
    return V;
    }

    // Return all edges in the graph
    auto &edges() const{
    return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
        e.dest >= 1 && e.dest <= V)
        edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
    std::vector<Edge<T>> edges_from_v;
    for (auto &e : edge_list)
    {
    if (e.src == v)
    edges_from_v.emplace_back(e);
    }
    return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:

    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;

};
```

ILO B2

Code Output

```
PS C:\Users\TIPQC\Documents\Barizo_CPE010\HOA10p1> & C:
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Code Snippet

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

#ifndef ILOB2_H
#define ILOB2_H

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const{
    return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const {
    return this->weight > e.weight;
    }

    void add_edge(Edge<T> &&e){}
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
    os << std::endl;
    }
    return os;
}

template <typename T>
class Graph {

public:
    Graph(size_t N) : V(N) {}
    auto vertices() const
```

```
public:
    Graph(size_t N) : V(N) {}
    auto vertices() const
    {
    return V;
    }
    auto &edges() const
    {
    return edge_list;
    }
    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
        std::cerr << "Vertex out of bounds" << std::endl;
    }

auto outgoing_edges(size_t v) const {
    std::vector<Edge<T>> edges_from_v;
    for (auto &e : edge_list) {
        if (e.src == v) {
            edges_from_v.emplace_back(e);
        }
    }
    return edges_from_v;
}

template <typename Q>
friend std::ostream &operator<<(std::ostream &os, const Graph<Q> &G);

private:

    size_t V;
    std::vector<Edge<T>> edge_list;

};

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
```

```
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
        for (auto e : G.outgoing_edges(current_vertex))
        queue.push(e.dest);
        }
}

return visit_order;
}

template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
```

```
return visit_order;
}

template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
    std::cout << v << std::endl;
}

#endif
```

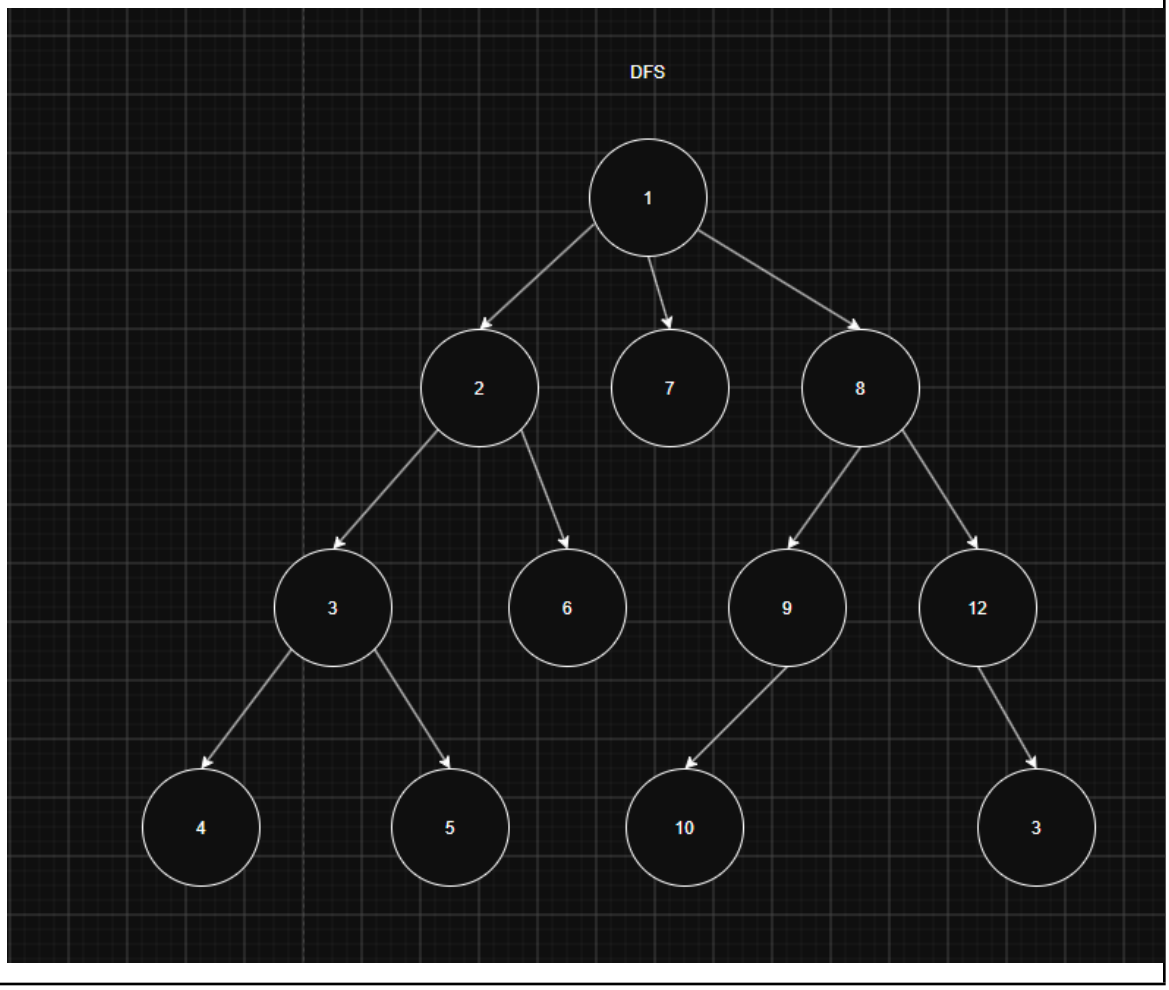
ILOB2.cpp > main()

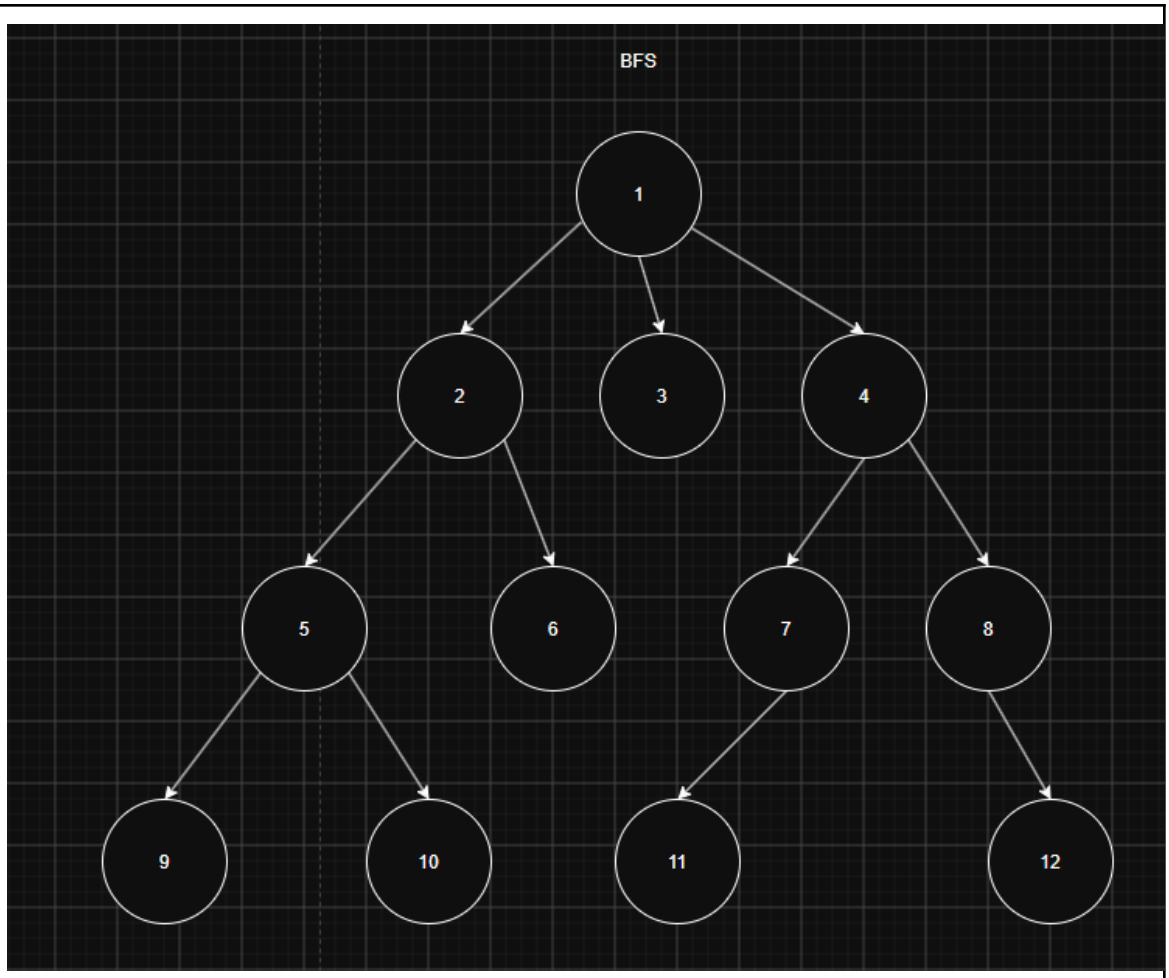
```
1   #include<iostream>
2   #include "ILOB2.h"
3
4   int main()
5   {
6   using T = unsigned;
7   test_BFS<T>();
8   return 0;
9   }
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1.) A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
A Depth First Search would be the most helpful algorithm in this task. Because a DFS explores a graph by going as deep as possible along a single path before reaching a dead end, at which point it backtracks to explore other unvisited adjacent vertices. Because the person moves continuously along connected locations until hitting a dead end and then backtracks to check alternative branches from the previous vertex, DFS naturally mirrors this continuous path exploration.
2.) Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

The tree equivalent of DFS is preorder traversal (a depth-first tree traversal). Both go as deep as possible along one path before backtracking, and both can be implemented recursively or with a stack. The difference is that trees have no cycles, so preorder doesn't need a visited set, while graph DFS does. The tree equivalent of BFS is level-order traversal, which uses a queue.





Pseudocode

Graph DFS

```
dfs(v):
   mark v as visited
   for each unvisited vertex u adjacent to v:
      dfs(u)
```

Tree Preorder (The Equivalent)

```
preorder(node):
   if node is null: return
   visit(node)           // Root
   preorder(node.left)     // Left
   preorder(node.right)    // Right
```

The only real difference is that a tree has no cycles, so no visited marking is needed. Each node has exactly

one parent, so you can never reach it twice.

Code implementation

```cpp
#include <iostream>
#include <stack>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

// Recursive preorder
void preorder(TreeNode* node) {
    if (node == nullptr) return;
    cout << node->val << " ";   // visit root
    preorder(node->left);       // go deep left
    preorder(node->right);      // then right
}

// Iterative preorder
void preorderIterative(TreeNode* root) {
    if (root == nullptr) return;
    stack<TreeNode*> s;
    s.push(root);
    while (!s.empty()) {
        TreeNode* cur = s.top();
        s.pop();
        cout << cur->val << " ";
        if (cur->right) s.push(cur->right);  // push right first
        if (cur->left)  s.push(cur->left);   // so left is popped first
    }
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    preorder(root);            cout << endl;
    preorderIterative(root);   cout << endl;
    return 0;
}
```

```
1 2 4 5 3
1 2 4 5 3
```

3.) In the performed code, what data structure is used to implement the Breadth First Search?
The data structure that was used to implement the BFS was a Queue Data Structure. It follows a first in, first out order to explore the vertices layer by layer.
4.) How many times can a node be visited in the BFS?
A node is visited or processed exactly 1 time. Once a vertex is dequeued and marked as visited in the visited data structure, the algorithm checks this record and skips it if encountered again from another neigbour, which avoids redundant processing and infinite loops.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Utilization of AI was used to generate ideas and guide me through the process and code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.